Technical report — BriefKlar

Official letters, explained in your own language. An Apertus-based assistant that turns Swiss administrative letters into checked deadlines and plain-language explanations, including in low-resource languages.

- **Track:** Track 2B — BriefKlar
- **Event:** Online
- **Team:** BriefKlar — Shijin Zhou
- **Demo:** https://drive.google.com/file/d/1fKHgZEVLGQA5CbuE2uhye68dvnFwYToS/view?usp=sharing
- **Dataset:** https://huggingface.co/datasets/kiki12111/briefklar-swiss-letters

1. Summary

Problem. Migrants, refugees and other residents who don't read German, French or Italian regularly receive official letters with hard deadlines: tax bills, fines, residence-permit renewals, insurance reminders, debt collection. Missing a deadline costs money or legal standing. Machine translation translates words, but it doesn't say what to do by when, and it is weak in many of the languages these people speak (Tigrinya, Somali, Kurdish.....).

Approach. BriefKlar uses Apertus to extract the letter's facts as JSON (dates, amounts, actions). Code then checks every date and amount against the letter text. Apertus writes an action-oriented explanation, and a separate Apertus call translates it into the user's language. The app also exports deadlines to a calendar, answers follow-up questions and drafts replies.

Headline result. On 12 synthetic Swiss letters (DE/FR/IT), Apertus v1.5 70B found **every deadline** and invented **no dates or amounts**. Explanations in English, Ukrainian, Tigrinya and Arabic kept 100% of deadlines and amounts. The main finding concerns language drift. Asked to write Tigrinya directly, Apertus wrote fluent **Amharic** in 9 of 12 cases, which date and amount metrics cannot detect. Writing in English first, then translating with the language also named in its own script (Ge'ez) brought correct-language output to 12 of 12.

2. Architecture

```
Browser (single static page, no external assets)
   │  letter text / PDF
   ▼
FastAPI app (Docker image `briefklar`)
   1. extract   — Apertus → JSON facts (temperature 0)
   2. ground    — code: every date/amount re-found in the letter text, else flagged
   3. explain   — Apertus → English explanation; code rewrites dates to DD.MM.YYYY
   4. translate — Apertus → "<Language> (<native name>)", e.g. Tigrinya + Ge'ez name
   5. extras    — Q&A, reply draft (Apertus); .ics calendar export (code)
   │
   ▼  OpenAI-compatible HTTP (LLM_BASE_URL, LLM_NAME, LLM_API_KEY)
Apertus: Hack Apertus endpoint, or vLLM with local weights (profile `local-gpu`)
```

The app keeps no state and writes no letters to disk. Its only outbound connection is `LLM_BASE_URL`.

Target architecture (mandatory)

Target: a) On-premise. BriefKlar runs entirely on the organisation's own infrastructure, for example a migrant counselling service, a municipality or a cantonal office, under its own administration:

- **Everything is containerised.** `make run-local` starts two containers on the organisation's own GPU host: the BriefKlar app, and vLLM serving Apertus v1.5 from local weights (`models/apertus`, downloaded once from Hugging Face). Docs: `DEPLOYMENT.md`.
- **Letters never leave the organisation.** The app's only outbound connection is `LLM_BASE_URL`, which on-premise points to the local vLLM container. The web page loads no CDN assets or fonts (a unit test checks this), Swagger UI is off, and no letter is written to disk.
- **Open weights make this possible.** Apertus can be self-hosted without a licence server or vendor API, so the organisation controls the model version, updates and logs.

The same setup also works offline (b), with offline flags already set in the compose file, and can be moved to a Swiss-operated cloud VM (c). We claim only (a).

External dependencies

	Build time	Runtime
Python packages (FastAPI, openai, pydantic, pypdf, ...)	PyPI or pre-downloaded wheels	—
Base images	`python:3.11-slim`, `vllm/vllm-openai`	—
Model weights	Hugging Face `swiss-ai/Apertus-v1.5-8B` / `-70B`	local files
Inference	—	`LLM_BASE_URL` only (on-premise: the local vLLM container)

During the hackathon, all results were produced through the Hack Apertus hosted endpoint (`make run`). The on-premise configuration only changes `LLM_BASE_URL`.

3. Use of Apertus

- **Model:** `apertus-v1.5-70b` for all reported results (Hack Apertus endpoint). `apertus-v1.5-8b` works too (smoke-tested); it is the default for local GPU serving.
- **How it is used:** inference. Four task-specific prompts: extraction, explanation, translation, and Q&A/reply. No fine-tuning.
- **Where it runs:** hosted OpenAI-compatible endpoint during the hackathon; vLLM with local weights for on-premise and air-gapped deployment.

Setup details (all in `src/app/`):

- `prompts.py`: extraction returns a fixed JSON schema and is told never to compute dates ("within 10 days" stays a rule, not a date). The explanation follows a fixed five-section structure at about CEFR A2 level and keeps original official terms in parentheses.
- `pipeline.py`: JSON is parsed tolerantly (code fences, surrounding prose) with one repair retry. Temperature is 0 for extraction and translation and 0.2 for explanations.
- `languages.py`: every prompt names the target language natively as well, e.g. Tigrinya together with its name in Ge'ez script. See finding 2 below.
- `grounding.py`: deterministic parsing of DE/FR/IT/EN date formats and Swiss number formats (`2'486.40`, `Fr. 120.–`), plus non-Latin digits.

Other models: none. No other LLM was used in the product or in the evaluation. The eval harness can call an optional judge model (`--judge`), but it was **not** used for any result in this report. The language check in the evaluation is a deterministic heuristic, not a model.

4. Data

- `data/letters.jsonl`: **12 synthetic letters written for this project** (7 German, 3 French, 2 Italian; 9 letter types: tax, fine, residence permit, health insurance, debt collection, employment office, school, housing, social services). They imitate the style of Swiss administrative mail. All names, addresses, institutions and reference numbers are **fictional**, so there is no personal data. Each letter has hand-written gold labels: dates, deadlines, amounts, letter type and required actions. One letter is a deliberate trap: a debt-collection order with only relative deadlines ("within 10 days"), where the correct output contains no date.
- `data/results/runs/`: the full model responses and scores for every evaluation run.
- Published on Hugging Face as test cases, model responses and metadata (instance-level licences): https://huggingface.co/datasets/kiki12111/briefklar-swiss-letters
- License: CC-BY-4.0 (created by the team).

5. Evaluation

Tasks.

1. *Extraction:* letter → structured facts. Scored against gold labels.
2. *Explanation:* facts and letter → plain-language explanation in a target language. Scored for (a) being in the requested language, (b) keeping every gold deadline and amount, and (c) following the required 5-section

structure. All checks are deterministic code in `src/eval/`.

The language check uses script detection. For Ge'ez script it additionally counts Tigrinya vs. Amharic marker words and the orthographic variant.

Runs: `data/results/runs/run-20261006-142305-pivot.*` and `run-20261006-141428-direct.*` (`apertus-v1.5-70b`, 12 letters × 4 languages).

5.1 Extraction

Setup	Metric	Result
Baseline: regex only (mock model)	date recall / precision	1.00 / 0.95
Baseline: regex only (mock model)	amount precision · letter type · action recall	0.18 · 0.00 · 0.00
Apertus 70B + grounding	deadline recall	**1.00**
	date recall / precision	0.91 / 1.00
	amount recall / precision	0.94 / 1.00
	letter type accuracy · action recall	0.92 · 0.98
	invented dates / amounts (code check)	**0 / 0**
	"no explicit date" trap	passed: no date invented; the 10-day rule is reported as an action

Regex finds dates but cannot tell which numbers matter, what kind of letter it is, or what to do. Apertus can. The code-level grounding check keeps the model honest on the numbers. The misses were secondary facts only: a permit expiry date, an offence date, a CHF 30 fee and the old rent.

5.2 Explanations: direct generation vs. English pivot + translation

Target language	In requested language: direct	In requested language: pivot	Deadlines & amounts kept: direct	Deadlines & amounts kept: pivot	Avg. seconds (direct → pivot)
English	1.00	1.00	1.00	1.00	6.3 → 6.4
Ukrainian	1.00	1.00	1.00	1.00	8.6 → 15.4
Tigrinya	**0.25**	**1.00**	1.00	1.00	31.3 → 43.4
Arabic	1.00	1.00	0.91	1.00	6.1 → 13.7

Structure compliance was 1.00 in all cells of the final runs. Extraction takes another ~6 s per letter.

5.3 Findings

1. **Silent drift into a related, higher-resource language.** With direct generation, 9 of 12 "Tigrinya" explanations were Amharic: same script, different language. Dates and amounts were all correct, so a fact-only metric gives full marks. A Tigrinya reader would get a polished but partly incomprehensible text. A language check is necessary when evaluating low-resource languages.
2. **Name the language in its own script.** A translation prompt that said only "Tigrinya" produced Oromo in an early run. Adding its native name in Ge'ez script gave Tigrinya in every trial. This is now applied to all prompts (`app/languages.py`).
3. **Pivot plus translation is the default.** It fixes the drift at the cost of one extra call, roughly 2× latency for non-English targets.
4. **Token cost of Ge'ez script.** A Tigrinya translation of a 1,306-character English text needed 1,919 output tokens. Our initial 1,500-token limit silently truncated 2 of 12 Tigrinya explanations, and one lost its deadline. The client now detects `finish_reason == "length"` and retries with a larger limit. Limits must be sized for the target script, not for English.
5. **Prompt examples leak.** An example term in the prompt, "(Einsprache)", appeared in an explanation of a letter that never used it. Examples were removed from the prompt.
6. **Format instructions are followed unevenly.** The model often wrote "November 30, 2026" instead of the requested DD.MM.YYYY. Dates are now rewritten in code.

6. Limitations

- **Small synthetic test set** (12 letters, written by the team). Real letters are longer, contain tables, and arrive as photos.
- **No OCR yet.** The app accepts text and PDFs with a text layer only.
- **The language check is a heuristic.** Script detection plus Tigrinya/Amharic marker words. Tigrinya outputs have not yet been reviewed by a native speaker.
- **Q&A and reply drafting generate directly** in the target language. For Tigrinya they may show the same drift as direct explanations (finding 1); they should use the translation step too.
- **Secondary facts are sometimes missed**, e.g. expiry dates and side fees. Deadlines were always found, but the UI should invite users to check the original letter.
- **Latency.** Pivot plus translation takes about 2× as long; a Tigrinya explanation takes about 43 s.
- **The direct-strategy run used the 1,500-token limit before the truncation fix.** Its language-drift result is unaffected, because drift is visible from the first sentence.
- **The on-premise path was not run end-to-end.** `make run-local` (vLLM on a local GPU) was checked for configuration and Makefile syntax only; no GPU was available during the hackathon. The app container itself was tested with `make run`, `make demo` and `make test`.
- **Not legal advice.** The UI says so and flags unverified facts.

7. Reproducibility

- **Run:** `cp .env.example .env`, set `LLM_API_KEY`, then `make run` (web app on :8000) or `make demo` (terminal, one letter end-to-end).
- **Numbers:** `make eval` runs both strategies over `data/letters.jsonl` and writes `data/results/runs/run-<timestamp>-<strategy>.{json,md}`.
- **Tests:** `make test` (offline, mock model).
- **Hardware:** any machine with Docker for the hosted-endpoint setup; an NVIDIA GPU with ≥24 GB VRAM for `make run-local` with the 8B model.
- **Settings:** `apertus-v1.5-70b`, temperature 0 (extraction, translation) / 0.2 (explanation). Max tokens are 1,500 by default and 4,096 for translation; any call cut off by the limit is retried once with a larger limit. The endpoint does not guarantee determinism, so expect small run-to-run variation.
- **Commit:** `d98b49c` on https://github.com/kiky97/briefklar (code verified from a fresh clone with `make test`, `make demo` and `make run`; later commits change only the report)

8. Next steps

- Native-speaker review of Tigrinya, Somali and Arabic outputs, and co-design of test letters with migrant counselling services
- Route Q&A and reply drafting through the translation step for low-resource languages
- OCR for photographed letters, running locally in the same container
- Compare Apertus v1.5 8B against 70B on cost per letter, to support small on-premise deployments
- A request queue for the endpoint's parallel-request limit

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

References

- Hack Apertus project template: https://github.com/HackApertus/project-template
- Apertus v1.5 models: https://huggingface.co/swiss-ai/Apertus-v1.5-8B, https://huggingface.co/swiss-ai/Apertus-v1.5-70B